

Lesson 13: More on Functions

In lesson 4 you were given the basic information on functions. However, I left out one item of interest. That item is the inline function. Inline functions are not very important, but it is good to understand them. The basic idea is to save time at a cost in space. Inline functions are a lot like a placeholder. Once you define an inline function, using the 'inline' keyword, whenever you call that function the compiler will replace the function call with the actual code from the function.

How does this make the program go faster? Simple, function calls are simply more time consuming than writing all of the code without functions. To go through your program and replace a function you have used 100 times with the code from the function would be time consuming not too bright. Of course, by using the inline function to replace the function calls with code you will also greatly increase the size of your program.

Using the inline keyword is simple, just put it before the name of a function. Then, when you use that function, pretend it is a non-inline function.

For example:

```
#include <iostream>

using namespace std;

inline void hello()
{
    cout<<"hello";
}
int main()
{
    hello(); //Call it like a normal function...
    cin.get();
}
```

However, once the program is compiled, the call to hello(); will be replaced by the code making up the function.

A WORD OF WARNING: Inline functions are very good for saving time, but if you use them too often or with large functions you will have a tremendously large program. Sometimes large programs are actually less efficient, and therefore they will run more slowly than before. Inline functions are best for small functions that are called often.

In the future, we will discuss inline functions in terms of C++ classes. However, now that you understand the concept I will feel comfortable using inline functions in later tutorials.